

OnamDelivery

Technical Architecture & Technology Stack

A “Swiggy for Flowers” marketplace for Kannur / Onam — vendors, customers, and delivery partners on one platform.

Repository github.com/AanandAB/OnamDelivery **API** onam-flowers-api.aanandab44.workers.dev

Admin onam-flowers-admin.pages.dev **Author** Aanand AB

1. Overview

OnamDelivery is a hyperlocal flower-delivery marketplace for the Onam season. It connects three sides of a marketplace — **customers** who order flowers, **vendors** who sell them, and **delivery partners** who carry them — under a single **company-owner** dashboard that keeps the platform operator in full control. Every order carries a guaranteed per-order profit for the platform, and delivery partners are paid per kilometre at a rate that always beats their fuel cost.

The system is **edge-first and serverless**: the entire backend is one Cloudflare Worker talking to a serverless SQLite database (D1), with no self-hosted servers, no containers, and no per-instance state.

Customers

Flutter

Browse, order, pay COD, live-track the partner, and rate the vendor.

Vendors

Web console

List stock, edit prices, toggle open & own-delivery, see the order queue.

Partners

Flutter · Android

Go online, accept orders, navigate with GPS, deliver with an OTP, earn per km.

Owner

Web dashboard

Edit any vendor’s stock/price, analytics, settlements, approvals, live ops map.

Key design goals — four role-isolated surfaces sharing one API · COD-first payments · vendor “zero-hassle” console · guaranteed platform margin · Indian DPDP Act 2023 compliance.

2. High-Level Architecture

Three client surfaces talk to a single stateless Worker API over HTTPS/JSON. The Worker is the only thing that reads or writes the database; clients never touch D1 directly.

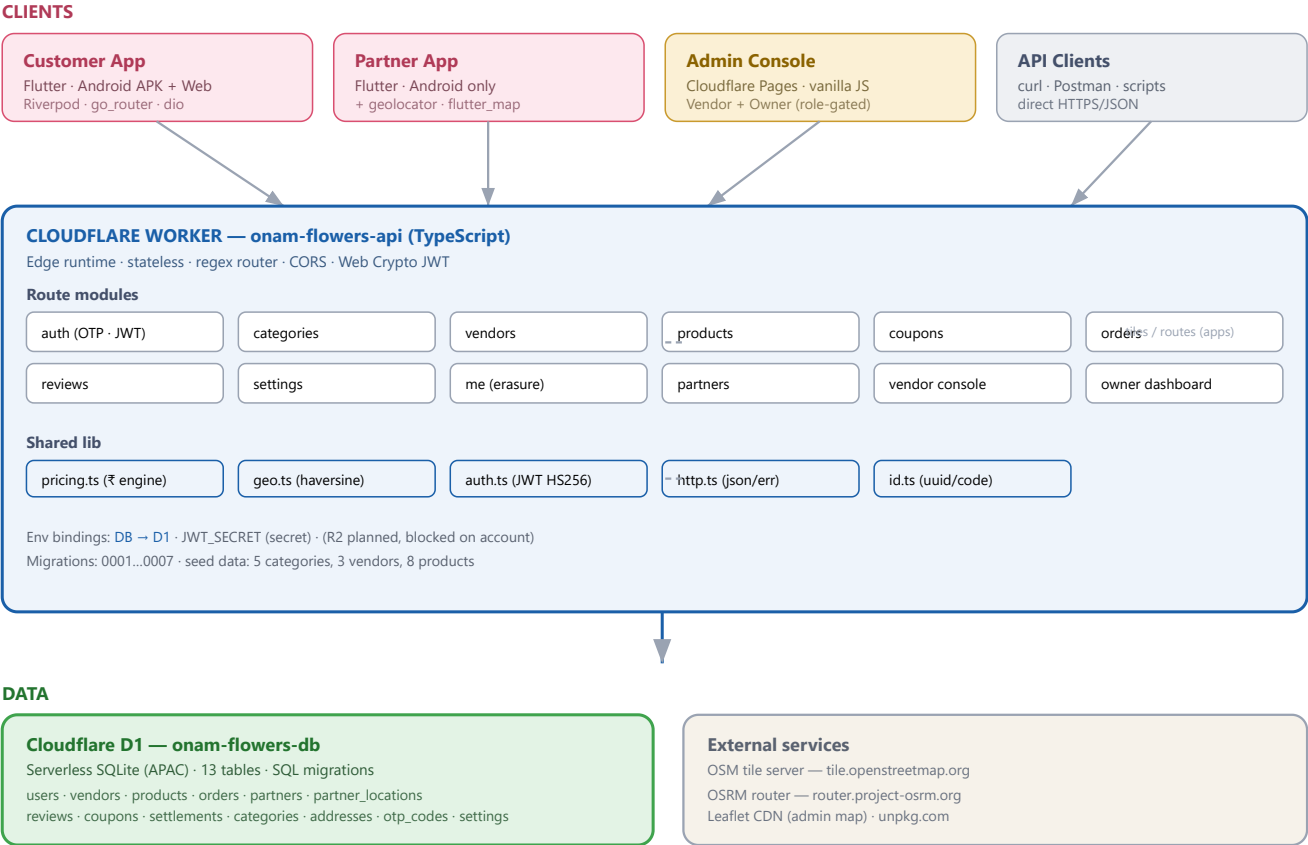


Figure 1 — System architecture. Four client surfaces converge on one Cloudflare Worker, which owns all data access via D1.

3. Technology Stack

3.1 Backend — Cloudflare Workers

COMPONENT	TECHNOLOGY	NOTES
Runtime	Cloudflare Workers	Serverless edge functions, V8 isolates, zero cold-start to speak of, deployed via Wrangler.
Language	TypeScript	Strictly typed; <code>tsc --noEmit</code> gate before every deploy.
Database	Cloudflare D1	Serverless SQLite at the edge; SQL migrations via <code>wrangler d1 execute</code> .
Router	Custom regex router	A <code>method + regex</code> dispatch table in <code>src/index.ts</code> — no framework dependency.

Auth	JWT (HS256)	Signed with Web Crypto API; <code>role</code> claim separates customer / partner / vendor / owner.
OTP	Phone OTP (dev)	Codes persisted in <code>otp_codes</code> and returned in the response for dev; SMS provider to be wired for production.
Object storage	R2 (planned)	Not yet enabled on the account; images currently use an <code>image_url</code> TEXT field.
CLI / deploy	Wrangler 4.x	<code>wrangler deploy</code> , <code>wrangler d1 execute</code> , <code>wrangler pages deploy</code> .

3.2 Admin console — Cloudflare Pages

COMPONENT	TECHNOLOGY	NOTES
Hosting	Cloudflare Pages	Single static <code>index.html</code> (no build step) deployed with Wrangler.
Frontend	Vanilla HTML / CSS / JS	Role-gated SPA — vendor console + owner dashboard in one file.
Maps	Leaflet 1.9.4	Ops map with OpenStreetMap tiles (vendors + online partners).
Auth	Reuses Worker JWT	Token in <code>localStorage</code> ; role chosen at login, sent as <code>Bearer</code> .

3.3 Customer app — Flutter

COMPONENT	TECHNOLOGY	NOTES
Framework	Flutter 3.47.1 / Dart 3.13.1	Targets Android APK + Web; Material 3.
State	flutter_riverpod 3.4	<code>Notifier</code> / <code>FutureProvider</code> / <code>family</code> for catalog & auth.
Routing	go_router 18	Declarative routes + <code>StatefulShellRoute</code> bottom-nav shell.
HTTP	dio 5.11	Bearer-token interceptor + typed API client.
Persistence	shared_preferences 2.5	JWT + profile persistence.
Maps	flutter_map 8.3 + latlong2	OSM tiles + OSRM route polyline for live order tracking.
Payments	url_launcher	UPI deep-link intent (<code>upi://pay...</code>) when the owner's VPA is configured.
UI	Neumorphism theme	Custom soft-UI design tokens (<code>core/theme.dart</code>).

3.4 Delivery partner app — Flutter

COMPONENT	TECHNOLOGY	NOTES
Framework	Flutter 3.47.1 / Dart 3.13.1	Android-only build (no web/Windows).
Everything above	Riverpod · go_router · dio · shared_preferences · flutter_map	Same stack as the customer app.
Location	geolocator 14	GPS permission + live position stream (10 m filter) → breadcrumbs to the API.
Routing	OSRM (driving)	Pickup→drop route geometry + distance + ETA.

3.5 Toolchain

TOOL	VERSION	ROLE
Flutter / Dart	3.47.1 / 3.13.1	Both apps.
Node.js	v24 LTS (active) · v26 via fnm	Wrangler + build scripts.
Wrangler	4.x	Worker + Pages + D1 deploys.
Android toolchain	AGP 9.1.0 · Kotlin 2.4.0 · Gradle 9.3.1	APK builds (Kotlin DSL).
VCS	Git + GitHub	Monorepo <code>onam-flutter</code> , main branch.

4. Data Model (D1)

Thirteen tables, seven migrations. Identity is phone-based; ids are UUIDv4 or short human-readable codes.

TABLE	PURPOSE	KEY COLUMNS
users	Customers	id, phone (unique), name, consented_at, consent_version
addresses	Saved delivery addresses	id, user_id, label, line1, lat, lng
vendors	Flower shops	id, name, phone, lat/lng, radius_km, is_open, has_own_delivery, rating, rating_count
categories	Product categories	id, name_en, name_ml, sort_order
products	Catalog items	id, vendor_id, category_id, name_en/ml, unit, price, stock, occasion, hidden
partners	Delivery fleet	id, phone, name, vehicle, kyc_status, is_online, current_lat/lng
partner_locations	GPS breadcrumbs	partner_id, lat, lng, ts

orders	Transactions	id, user_id, vendor_id, partner_id, status, items(JSON), subtotal, delivery_fee, delivery_pay, platform_fee, vendor_payout, total, delivery_type, distance_km, pickup/drop coords, otp
reviews	Ratings	id, order_id, vendor_id, user_id, rating, comment
coupons	Discount codes	id, code, type, value, phone, used
settlements	Vendor payout ledger	id, vendor_id, order_id, amount, status
otp_codes	Login codes	phone, code, expires_at, used
settings	Config / economics	key, value (rates, owner_phone, upi_id, grievance officer)

5. Authentication & Authorization

All four roles authenticate with **phone + one-time password**, then receive a JWT (HS256, 30-day expiry) carrying a `role` claim and the actor's id in `sub`. Consent (DPDP Act 2023) is mandatory and recorded on login.

ROLE	JWT CLAIM	LOGIN ENDPOINT	ACCESS
Customer	<code>sub = user id</code>	<code>/api/auth/*</code>	Own orders, reviews, tracking, erasure.
Partner	<code>role = "partner"</code>	<code>/api/partner/*</code>	Own profile, order discovery/claim/fulfilment, earnings.
Vendor	<code>role = "vendor"</code>	<code>/api/vendor/*</code>	Own catalog (price/stock), own orders, shop toggles.
Owner	<code>role = "owner"</code>	<code>/api/owner/*</code>	Everything: any vendor's catalog, analytics, settlements, approvals, settings.

Role isolation is enforced server-side by four guards — `requireAuth`, `requirePartner`, `requireVendor`, `requireOwner` — each rejecting tokens with the wrong `role`. A vendor can edit only their own products (ownership check returns 404 otherwise); a customer or partner token is rejected with 401 on any admin endpoint.

6. Economics & Pricing Engine

Every order's money is computed in `lib/pricing.ts` from rates stored in the `settings` table — so the owner can retune the whole platform without a redeploy. Rates derive from the fuel constraint: **₹116/litre ÷ 20 km/litre = ₹5.80/km**, so partner pay must always clear that.

SETTING	DEFAULT	MEANING
platform_fee	₹20	Flat developer profit on every order (guaranteed).
delivery_base_fee	₹40	Customer delivery base charge.

delivery_rate_per_km	₹15	Customer delivery charge per km.
partner_base_pay	₹30	Partner base earnings.
partner_rate_per_km	₹10	Partner earnings per km (beats ₹5.80 fuel → ≈₹4.2/km net).
fuel_cost_per_km	₹5.80	Reference for the above invariant.

Per order (platform delivery): `total = subtotal + delivery_fee + platform_fee` ; the vendor keeps `subtotal` , the partner earns `delivery_pay` , and the platform keeps `platform_fee + (delivery_fee - delivery_pay)` . On vendor self-delivery, `delivery_pay = 0` but the ₹20 platform fee still applies.

7. Order Lifecycle & Data Flow

An order moves through a linear fulfilment ladder, with a handover OTP at the final step.



Stock decremented & money computed at placement · partner's live position surfaced to the customer at /track · duplicate accept & wrong OTP rejected server-side.

Figure 2 — Order lifecycle, from placement to review.

7.1 End-to-end flow

- Browse** — the customer app fetches vendors (distance/radius-filtered) and products from the Worker.
- Order** — `POST /api/orders` validates stock, computes the ₹ breakdown, decrements stock, and stores the handover OTP.
- Assign** — a partner sees the order in `/api/partner/orders/available` and atomically claims it.
- Track** — the partner streams GPS to `/api/partner/me` ; the customer polls `/api/orders/:id/track` (5 s) to watch the partner move.
- Deliver** — the partner enters the customer's OTP to complete the delivery.
- Review** — the customer rates the order; the rating rolls into the vendor's average.
- Observe** — the owner dashboard reflects revenue, platform profit, partner earnings, and per-vendor settlements in real time.

8. API Surface

All endpoints return JSON with CORS enabled. Grouped by consumer.

GROUP	ENDPOINTS
Health / settings	<code>GET /health</code> · <code>GET /api/settings</code>
Catalog (public)	<code>GET /api/categories</code> · <code>GET /api/vendors</code> · <code>GET /api/vendors/:id</code> · <code>GET /api/vendors/:id/products</code> · <code>GET /api/vendors/:id/reviews</code> · <code>GET /api/products</code>
Customer auth	<code>POST /api/auth/otp</code> · <code>POST /api/auth/verify</code> · <code>DELETE /api/me</code>

Customer orders	POST /api/orders · GET /api/orders · GET /api/orders/:id · GET /api/orders/:id/track · POST /api/orders/:id/review
Coupons	POST /api/coupons · POST /api/coupons/validate
Partner	POST /api/partner/otp verify · GET PATCH /api/partner/me · GET /api/partner/orders[available] · POST /api/partner/orders/:id/accept status
Vendor console	POST /api/vendor/otp verify · GET PATCH /api/vendor/me · GET POST /api/vendor/products · PATCH /api/vendor/products/:id · GET /api/vendor/orders
Owner dashboard	POST /api/owner/otp verify · GET /api/owner/overview · GET POST /api/owner/vendors · PATCH /api/owner/vendors/:id · GET POST /api/owner/vendors/:id/products · PATCH /api/owner/products/:id · GET /api/owner/orders partners settlements settings · PATCH /api/owner/partners/:id settings

9. Compliance (Indian regulations)

DPDP Act 2023

- **Explicit, recorded consent** — every login (customer, partner, vendor, owner) requires a consent checkbox; the API records `consented_at` + `consent_version` and refuses (400) without it.
- **Right to erasure** — `DELETE /api/me` anonymizes the user's personal data and deletes addresses.
- **Grievance officer** — published in `/api/settings` and shown in the customer app.
- **Purpose limitation** — partner/vendor consent text states exactly what data is processed (name, phone, location) and why.

Consumer Protection (E-Commerce) Rules 2020

- **Transparent pricing** — item price, delivery fee, and the ₹20 platform fee are itemized at checkout and in the order history.
- **COD-first** — cash on delivery is the default payment method; UPI is an optional extra.

10. Deployment Topology

ARTIFACT	WHERE	NOTES
Backend API	<code>onam-flowers-api.aanandab44.workers.dev</code>	Cloudflare Worker; every commit → <code>wrangler deploy</code> .
Database	D1 <code>onam-flowers-db</code>	APAC region; SQL migrations applied via Wrangler.
Admin console	<code>onam-flowers-admin.pages.dev</code>	Cloudflare Pages (<code>wrangler pages deploy</code>).

Customer app	Flutter build	Android APK + web build (no store distribution yet).
Partner app	Flutter build	Android APK (debug builds verified).
Source	github.com/AanandAB/OnamDelivery	Monorepo; main branch.

Repository layout

```

onam-flutter/
├── apps/
│   ├── customer_app/    # Flutter - Android + web
│   └── delivery_app/    # Flutter - Android only
├── backend/
│   ├── worker/          # Cloudflare Worker (TypeScript)
│   │   ├── migrations/  # 0001...0007 SQL
│   │   ├── seed.sql
│   │   └── src/
│   │       ├── index.ts    # router
│   │       ├── env.ts      # bindings
│   │       ├── lib/        # pricing, geo, auth (JWT), http, id
│   │       └── routes/     # auth, categories, vendors, products, coupons,
│   │                           # orders, reviews, settings, me, partners, vendor, owner
│   └── admin/            # Cloudflare Pages - index.html (vendor + owner console)
└── docs/                 # PLAN.md, API.md, ARCHITECTURE.*

```

11. Current State & Production Roadmap

Shipped & verified — all four surfaces; 4-role auth; full economics; DPDP compliance; live tracking; reviews; UPI intent; ops map. Backend flows verified with 53+ scripted checks; both apps build clean.

ITEM	STATUS / PLAN
SMS OTP provider	Dev mode returns OTP in the response; wire an SMS gateway (MSG91 / 2Factor) before real customers.
JWT secret	Dev fallback in <code>auth.ts</code> ; set a real one via <code>wrangler secret put JWT_SECRET</code> .
R2 image storage	Blocked — R2 not enabled on the Cloudflare account; images use <code>image_url</code> strings.
UPI gateway	Intent deep-link works; add Razorpay (or an RBI-licensed aggregator) for server-verified UPI + webhooks.
Map tiles / routing	Public OSM tiles + OSRM demo server are fine for dev; self-host or license for production scale.
Push notifications	Not yet integrated; needs FCM/OneSignal + server credentials.
App store distribution	APK/web builds exist; signing + Play Store listing pending.

